CPSY 1291 · RECITATION HANDOUT 3

Matrices, curve fitting, and methods with knobs

The week before Assignment 1 is due

Fall 2026 · Week 3 · hold this after Lecture 4, before Tue 9/29

What this session is for

Recitations 1 and 2 gave you arrays, shapes, distances, dissimilarity matrices and PCA. Assignment 1 needs four more things, and none of them is taught in lecture:

1. **matrix multiplication** as an operation you can predict the shape and the meaning of, rather than a symbol you try until it runs;
2. **fitting a curve to data** and comparing two candidate curves honestly (Part 2 asks you to test Shepard's law);
3. **rank correlation**, and why comparing two dissimilarity matrices uses it rather than Pearson (Part 2k);
4. **methods with knobs** — t-SNE's perplexity, every method's `random_state` — and how to report a result that depends on them (Part 4).

> **The rule this week.** A method with a knob does not give you a result. It gives you a *family* of results, one per knob setting. Your job is to say which features of that family survive the knob and which do not — and only the survivors are findings.

1 Matrix multiplication, and how to predict it

1.1 The shape rule, and why it is the whole thing

If $\boldsymbol{A}$ is $(m \times k)$ and $\boldsymbol{B}$ is $(k \times n)$, then $\boldsymbol{AB}$ is $(m \times n)$. The inner dimensions must match and they *vanish*; the outer dimensions survive.

```
A = np.zeros((120, 4096)) # 120 images x 4096 units
B = np.zeros((4096, 10)) # 4096 units x 10 categories
(A @ B).shape # (120, 10) -- 4096 cancelled
```

Say the cancelled dimension out loud: "*summed over units*". That is what a matrix product is — a sum over the shared index — and naming it tells you immediately whether the operation means anything.

> **`@` is not $*$.** `A * B` multiplies element by element and broadcasts. `A @ B` contracts the shared index. They agree on nothing, and NumPy will happily run the wrong one when the shapes permit. If you wrote $*$ and got an answer with the shape you expected,

check that you did not mean `@`.

1.2 The three products you will actually write

Let $\boldsymbol{X}$ be $(n \times p)$: n stimuli down the rows, p features across the columns. Then

Expression	Shape	What it holds
`X @ X.T`	$(n \times n)$	every pair of *stimuli*, dotted
`X.T @ X`	$(p \times p)$	every pair of *features* $\rightarrow$ covariance
`X @ v`	$(n,)$	every stimulus projected onto direction $\boldsymbol{v}$

The first is where dissimilarity matrices come from. The second, after centering and dividing by $n-1$, *is* the covariance matrix whose eigenvectors PCA returns. The third is what `pca.transform` does, once per component. Three lines, and they are most of the assignment.

1.3 Transpose, and the mistake it causes

`X.T` flips rows and columns. The failure mode is not that it errors — it is that on a square matrix it does not, and you silently compute the covariance of your stimuli instead of your features. Whenever you write `.T`, write the resulting shape in a comment. It costs three seconds and it is the single highest-yield habit in this course.

> **Order matters.** $\boldsymbol{AB} \neq \boldsymbol{BA}$, even when both are defined. Matrix multiplication composes operations, and composing in the other order is a different operation — rotate-then-stretch is not stretch-then-rotate.

2 Reading an eigenvalue spectrum

Recitation 2 said what an eigenvector is: a direction the covariance matrix stretches without rotating, and PC1 is the most-stretched one. Here is the part you need for Part 3.

`pca.explained_variance_ratio_` is not a number, it is a **curve**: eigenvalue 1, eigenvalue 2, ...each divided by their total. Sorted, always decreasing. What the curve looks like is the finding.

```
ratio = pca.explained_variance_ratio_
plt.plot(np.cumsum(ratio)) # how much you have kept by component k
plt.axhline(0.9, ls=':') # a threshold you must choose and defend
```

- A spectrum that **falls off a cliff** means a genuinely low-dimensional representation: a handful of directions carry it.
- A spectrum that **decays slowly** means variance is spread across many directions, and no small number of components summarizes it.

- "k components explain 90%" is meaningless without saying *out of how many*, and *on how many items*.

That last point is the trap. If your stimuli are 51 objects photographed at 24 angles each, rotating one object traces a smooth curve through feature space: you have added $24\times$ the points but hardly any new directions. The representation looks compact, and the number changes if you change the number of viewpoints while the network stays byte-for-byte identical.

> **Dimensionality is a property of the data *and* the stimulus set.** Never report it as a property of the network alone.

3 Fitting a curve, and comparing two

3.1 The two calls

For a polynomial, NumPy is enough:

```
c = np.polyfit(x, y, deg=1) # coefficients, highest power first
yhat = np.polyval(c, x)
```

For anything else — an exponential, a Gaussian — you write the function and let SciPy find the parameters:

```
from scipy.optimize import curve_fit

def expo(d, a, b): return a * np.exp(-b * d)

p, _ = curve_fit(expo, d, s, p0=[1.0, 1.0]) # p0 = a starting guess
yhat = expo(d, *p)
```

`curve_fit` minimizes squared error by local search, so it needs a starting guess and can land in a bad local minimum. If a fit looks absurd, try a different `p0` before you conclude anything about the data.

3.2 Comparing candidate curves

$$R^2 \;=\; 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

```
def r2(y, yhat):
   return 1 - ((y - yhat)**2).sum() / ((y - y.mean())**2).sum()
```

Three cautions, all of which bite in Part 2:

1. **R^2 is a ratio, so the denominator matters.** Bin your data before fitting and R^2 rises with the fitted curve unchanged — you removed noise from the denominator, not error from

the model. Two R^2 values are comparable only against the same target.

2. **More parameters never fit worse.** A curve with three free parameters will beat one with two on the same data, always. That is arithmetic, not evidence. Compare models with the *same* number of parameters, or say plainly that you did not.

3. **An R^2 can be negative**, if your fit is worse than predicting the mean. This is a real outcome, not a bug.

> **What Shepard's law claims.** That perceived similarity falls *exponentially* with distance in psychological space — $s = a\,e^{-bd}$, not $s = a - bd$ and not a Gaussian. Testing it means fitting all three and comparing, which is exactly the machinery above.

4 Pearson, Spearman, and the upper triangle

4.1 Which correlation, and why

Pearson measures how close the relationship is to a straight line. **Spearman** replaces every value by its rank and then computes Pearson on the ranks, so it measures whether the relationship is *monotonic* — when one goes up, does the other?

```
from scipy.stats import pearsonr, spearmanr
r, p = pearsonr(a, b)
rho, p = spearmanr(a, b)
```

Comparing two dissimilarity matrices uses Spearman, and the reason is substantive. A network's distances and a human's similarity judgments are not on the same scale and there is no reason to expect a linear relation between them. What you want to ask is: *do the two systems order the pairs the same way*? That question is Spearman's, and Pearson would answer a different one and report a lower number for a reason that has nothing to do with the models.

4.2 Never correlate the whole matrix

A dissimilarity matrix is symmetric with a zero diagonal, so more than half its entries are duplicates or structural zeros. Feeding all n^2 of them into a correlation inflates the sample and counts every pair twice. Take the upper triangle:

```
iu = np.triu_indices(D1.shape[0], k=1) # k=1 skips the diagonal
rho, p = spearmanr(D1[iu], D2[iu])
```

> **The p-value here is wrong, and you should say so.** The $n(n-1)/2$ pairs are not independent — they are built from only n stimuli, so moving one stimulus moves a whole row and column at once. The correlation is meaningful; its p-value is not, and the honest move is to report the correlation and note the dependence. (The standard fix is a permutation test over *stimuli*, which you will meet in Recitation 8.)

5 Methods with knobs

5.1 `random_state`: reproducible is not the same as meaningful

MDS and t-SNE both start from a random initialization and optimize. Two runs give two different pictures.

```
Y = TSNE(n_components=2, perplexity=30, random_state=0).fit_transform(X)
```

Setting `random_state` makes the run **reproducible**: you will get the same picture tomorrow. It does not make the picture **meaningful**: you have picked one member of a family and hidden the rest. The way to tell the difference is to run several seeds and look at what stays put.

> **The test to apply.** Rerun with three different seeds. Structure that appears in all three is a candidate finding. Structure that appears in one is a property of that seed, and reporting it is an error — a reproducible one.

5.2 Perplexity, and what a t-SNE map is not

t-SNE's `perplexity` sets roughly how many neighbors each point tries to stay near — small values preserve local detail, large values preserve coarser structure. Sweep it and the picture changes substantially.

Three things a t-SNE map does *not* tell you, all of which students read off it anyway:

- **Distances between clusters are not distances.** Two clusters drawn far apart are not more different than two drawn close together.
- **Cluster sizes are not sizes.** The algorithm expands sparse regions and compresses dense ones by design.
- **Apparent clusters are not evidence of clusters.** t-SNE will produce well-separated blobs from data with no cluster structure at all, at some perplexities.

What it *is* good for: seeing whether points that you already have labels for land together. That is a legitimate and useful question, and it is essentially the only one the map answers reliably.

5.3 What two dimensions cost

Any method that puts high-dimensional data on a page must throw something away. The question worth asking — and the one Part 4 asks — is what. A usable measure is **neighborhood preservation**: for each point, what fraction of its k nearest neighbors in the original space are still among its k nearest neighbors in the embedding?

```
def neighbor_overlap(D_hi, D_lo, k=10):
   n = D_hi.shape[0]
   hi = np.argsort(D_hi, axis=1)[:, 1:k+1] # skip self at position 0
```

```
    lo = np.argsort(D_lo, axis=1)[:, 1:k+1]
    return np.array([len(set(hi[i]) & set(lo[i])) / k for i in range(n)])
```

This returns one number per point, so you can also *map* it — color each point by how well its neighborhood survived — and see which regions of the embedding you are allowed to trust.

6 A recipe for when it breaks

1. **Read the last line of the traceback first.** It names the error; everything above it is the route taken to get there.
2. **Then find the last line that is *your* file.** The frames below it are inside a library and are almost never the bug.
3. **Print shapes, not values.** `print(X.shape, Y.shape)` resolves most of them outright.
4. **Check for `nan` early.** `np.isnan(X).sum()` — one `nan` propagates through a whole matrix silently. Correlation distance produces them for any constant row (a dead unit), which is why Part 2c makes you look.
5. **Shrink the input.** Run on the first 5 rows. If the bug survives, you can now inspect every number by hand; if it does not, it is about scale, not logic.

7 Exercises

1. `X` is $(120, 4096)$ and `W` is $(4096, 64)$. What are the shapes of `X @ W`, `W.T @ X.T`, and `X @ X.T`? Which one is a dissimilarity-matrix-shaped object?

2. You compute $R^2 = 0.84$ for an exponential fit. A paper reports 0.96 for the same law on the same kind of data. Name two things that could differ between you and the paper that would produce this gap *without* either model being better.

3. You compare a network RDM with a human RDM and get Spearman $\rho = 0.42$, $p < 10^{-9}$ over $7{,}140$ pairs. What is wrong with the p-value, and what would you report instead?

4. A t-SNE map shows two clean, widely separated clusters. Give two different explanations, one of which is a real finding and one of which is not, and state the single check that separates them.

5. A colleague reports that a network's representation is "8-dimensional, since 8 components explain 90% of the variance." Their stimulus set is 40 objects at 30 viewpoints. What is the objection?

Answers

1. $(120, 64)$; $(64, 120)$; $(120, 120)$. The last one — it is square and indexed by stimuli on both sides.

2. (i) They may have *binned* the data before fitting, which shrinks the denominator of R^2 without improving the fit. (ii) They may have fitted a model with more free parameters. (Also acceptable: a different stimulus set with more spread in y, or averaging over subjects before fitting.)

3. The $7{,}140$ pairs come from only 120 stimuli and are therefore not independent, so the p-value's assumption fails and the number is far too small. Report ρ with the number of *stimuli*, and get a p-value from a permutation test that shuffles stimulus labels.

4. Real: the two groups genuinely occupy separate regions of the high-dimensional space. Not real: t-SNE manufactured the separation at this perplexity, or this seed. The check: rerun across several seeds *and* several perplexities, and see whether the split survives.

5. $1{,}200$ images, but only 40 distinct objects; the 30 viewpoints of one object trace a smooth low-dimensional curve rather than adding independent directions. The 90% figure is partly a fact about how the stimulus set was built, and it would move if they photographed 10 viewpoints instead — with the network unchanged.

Cheat sheet

You want	You write
every pair of stimuli, dotted	`X @ X.T`
covariance of the features	`np.cov(X, rowvar=False)`
project onto a direction	`X @ v`
how much each component explains	`pca.explained_variance_ratio_`
fit a line	`np.polyfit(x, y, 1)`
fit your own function	`curve_fit(f, x, y, p0=...)`
goodness of fit	`1 - sse / sst`
monotonic agreement	`spearmanr(a, b)`
upper triangle of an RDM	`np.triu_indices(n, k=1)`
same picture twice	`random_state=0`
count the `nans`	`np.isnan(X).sum()`
k nearest neighbors of each row	`np.argsort(D, axis=1)[:, 1:k+1]`